

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members (max 3):

Bayo Bandele (CruzID: 2141210)

Hegen Chang (CruzID: 2164517)

Spring 2026

1 Introduction

The goal of this project is to build a 100-class image classifier using transfer learning on a limited dataset. The dataset consists of 100 categories sampled from multiple sources, with approximately 10 training images per class and 1,036 test images. Because of the very small training set, training a model from scratch would result in severe underfitting, as the model would simply not have enough data to learn meaningful features. Transfer learning is appropriate here because it allows us to leverage a model pre-trained on millions of images (ImageNet), which already understands general visual patterns such as edges, textures, and shapes. We fine-tune this pretrained model on our specific 100 classes, enabling fast convergence and reasonable accuracy despite the limited data. Our final approach used EfficientNet-B2 as the backbone, achieving 60% validation accuracy and a Kaggle public leaderboard score of 0.60.

2 Dataset

1. 100 classes total. Each class contains approximately 10 training images, giving roughly 1,000 training samples. We used an 80/20 train/validation split (seed = 42), resulting in approximately 800 training samples and 200 validation samples. The test set contains 1,036 unlabeled images.
2. Training images are organized under train/0/ through train/99/. A critical issue encountered was that Python's default `os.listdir()` sorts folders alphabetically, causing folder '10' to map to label 2 instead of label 10. This was fixed by explicitly sorting with `key=lambda x: int(x)`, ensuring folder '0' maps to Label 0, folder '1' to Label 1, and so on, matching labels 0–99 to folders correctly.
3. All images were resized to 224x224. Normalization used ImageNet mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. Training augmentations included random horizontal flip, random vertical flip, random rotation (20 degrees), color jitter (brightness, contrast, saturation 0.4), and random grayscale (p=0.1). Validation used only resize and normalization.

3 Implementation

3.1 Model

4. EfficientNet-B2 with ImageNet pretrained weights (EfficientNet_B2_Weights.IMAGENET1K_V1). EfficientNet-B2 was chosen because it offers a strong accuracy-to-parameter tradeoff: larger than B0 for better feature extraction, but small enough to avoid severe overfitting on our 1,000-sample dataset.
5. The original EfficientNet-B2 classifier outputs 1,000 classes. We replaced the final linear layer with a new linear layer outputting 100 classes, matching our dataset.
6. All layers were unfrozen and fine-tuned end-to-end. A low learning rate ($2e-4$) with weight decay ($1e-3$) and a cosine annealing scheduler was used to avoid destroying the pretrained features.

3.2 Training

7. CrossEntropyLoss with label smoothing (0.1) to reduce overconfidence. AdamW optimizer with learning rate $2e-4$ and weight decay $1e-3$.
8. Batch size: 32. Epochs: 20. Learning rate scheduler: CosineAnnealingLR over 20 epochs. The best model checkpoint was saved based on validation accuracy.
9. Trained on Google Colab with a T4 GPU, using PyTorch with TorchVision, Python 3.10.

4 Experiments

10. Our initial baseline used EfficientNet-B0 with default ImageNet label ordering (alphabetic sort), achieving a misleading Kaggle score of 0.163 due to the label mapping bug described in Section 2.
11. Validation accuracy on the 20% held-out split was used as the primary tuning metric. We tuned the learning rate ($1e-3$ vs $2e-4$), the number of epochs (15 vs 20), and model size (B0 vs B2). CosineAnnealingLR replaced StepLR for smoother learning rate decay.
12. The most significant ablation was fixing the label mapping bug (alphabetic vs. numeric folder sorting). Without the fix, the Kaggle score was 0.163 despite seemingly reasonable training behavior. After the fix and a full pipeline rebuild, the Kaggle score improved to 0.60. Upgrading from B0 to B2 also provided a modest improvement in validation accuracy.

5 Results

13. Final training accuracy reached approximately 96.5% while validation accuracy plateaued at approximately 58–60%. This large generalization gap is expected given only 10 training images per class.
14. Public Kaggle leaderboard score of 0.60000 (60%), placing rank 30. This meets the 60% baseline threshold required for the full Kaggle portion of the project score.
15. Submission history: the first submission scored 0.163 (label mapping bug). The second submission scored 0.145 (partial fix, with residual random_split inconsistency). The third submission scored 0.600 after a full pipeline rebuild with correct numeric label ordering.

6 Discussion

16. Using a pretrained EfficientNet-B2 backbone was the most impactful decision. The pretrained features provided strong general representations that transferred well to our 100 classes. Heavy data augmentation also helped reduce overfitting given the small dataset.
17. Significant overfitting was observed throughout training: training accuracy reached approximately 96% while validation stayed near 58–60%. This is fundamental to the low-data regime, since with only 10 images per class, even a pretrained model with strong augmentation struggles to fully generalize.
18. The main limitation is dataset size. Next steps include Test-Time Augmentation (averaging predictions across multiple augmented versions of each test image), trying a stronger backbone such as EfficientNet-B4, gradually unfreezing layers during fine-tuning, and pseudo-labeling to synthetically expand the training set, with the goal of pushing the Kaggle score above 80%.

7 Reproducibility

19. A fixed seed of 42 was used for random, numpy, torch, and cuda. The train/validation split used `torch.Generator().manual_seed(42)`.
20. Environment: Google Colab with a T4 GPU, Python 3.10, PyTorch 2.x, and TorchVision 0.x. Dependencies can be installed with `pip install torch torchvision`.
21. To train, run all cells in the Colab notebook sequentially. To generate `submission.csv`, run the inference cell, which loops through `data/test/`, generates predictions for all 1,036 images, and downloads `submission.csv`.

8 Team Contributions

22. Bayo Bandele: dataset setup and loading pipeline, label mapping bug diagnosis and fix, Kaggle submission generation and format debugging, training execution.
23. Hegen Chang: model architecture selection and configuration, training loop implementation, hyperparameter tuning, presentation slide design.

9 References

24. TorchVision Models documentation: <https://pytorch.org/vision/stable/models.html>
25. Tan, M. and Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.
26. PyTorch Documentation: <https://pytorch.org/docs/stable/index.html>